Feature: Book a pet-care service

About this sample

This specification describes the service-booking flow in the Week 4, Lesson 8 PetCareAppMVC application. It documents the current database-backed behaviour: creating a booking and pending payment record does not process a payment.

The sample scenario uses Clara, her cat Mira, Basic Bath, and 3 November 2026. The date is valid only when it is today or a future date at submission time.

1. User goal

As a signed-in pet owner, I want to choose one of my pets, an available service, and a booking date so that I can create a booking and review its status.

The app records the booking and its payment as pending. Booking a service does not charge the customer or confirm that payment has been made.

2. Scope

Included

- Require the customer to be signed in to view services, create a booking, and view Booking History.
- List the customer's own pets and currently active services with prices.
- Allow the customer to select one pet, one service, and a valid booking date.
- Verify pet ownership and service availability when saving the booking.
- Save the booking and its pending payment record in one database transaction.
- Preserve the service price at booking time as the booking price.
- Show the customer's booking and payment statuses in Booking History.
- Display validation, unavailable-item, and database error outcomes.

Not included

- Real payment processing, payment-provider integration, or collection of payment credentials.
- Confirming a booking based on payment-provider approval.
- Cancelling or rescheduling bookings.
- Appointment time slots, capacity checks, or staff assignment.
- Booking a pet owned by another customer.

3. User flow

1. The customer opens the service-booking page.
2. The app lists pets owned by that customer and active services with their current prices.
3. The customer selects a pet, service, and date, then submits the form.
4. The app validates the submitted values and verifies pet ownership and service availability.
5. The app creates a `pending` booking and a related `pending` payment record within one database transaction.

6. On success, the app displays a success message and redirects the customer to Booking History.
7. Booking History displays the date, pet, service, booked price, booking status, and payment status.

If validation fails, the app displays an error and does not create records. If the pet or service is unavailable, the app rejects the booking. If a database error occurs, the app logs the error and does not report success.

4. Requests

Action	HTTP method	Path	Login
View available services and booking form	`GET`	`/services`	Required
Create a booking	`POST`	`/services/book`	Required
View booking history	`GET`	`/bookings`	Required

An optional `petId` query parameter can preselect a pet on the service page:

```
GET http://localhost:3000/services?petId=5
```

The pet is preselected only when it belongs to the signed-in customer.

The booking form submits these fields using URL-encoded form data:

Field	Required	Specification
`petId`	Yes	Positive integer ID of a pet owned by the signed-in customer
`serviceId`	Yes	Positive integer ID of an active service
`bookingDate`	Yes	Real calendar date in `YYYY-MM-DD` format; today or later

The client does not choose the price. The app obtains the price from the active service in the database and saves it as the booking's `booked_price`.

5. Available pets and services

The pet list includes only pets owned by the signed-in customer. The service list includes only services whose `is_active` value is `1`, ordered by name. Each listed service displays its price in Singapore dollars.

The sample database includes Clara Ng (username `clara`) and her cat Mira (pet ID `5`). It also includes Basic Bath, priced at S$25.00. These sample records can be used to exercise the example scenario; IDs are dependent on the database seed data.

If the customer has no pets, the page prompts them to add one. If there are no active services, the page explains that no services are currently available.

6. Booking and payment records

For every accepted booking, create one row in `service_bookings` and one related row in `payments`.

Record	Field	Required value
Booking	`pet_id`	The selected pet owned by the customer
Booking	`service_id`	The selected active service
Booking	`booking_date`	The valid date submitted by the customer
Booking	`booked_price`	The selected service's price at booking time
Booking	`booking_status`	`pending`
Payment	`booking_id`	The newly created booking ID
Payment	`amount`	The booking's `booked_price`
Payment	`payment_status`	`pending`

The two records are created in a single transaction. If booking creation, payment creation, or transaction commit fails, the transaction is rolled back; the app must not show a success message.

Creating these records does not contact a payment provider, charge the customer, or change either status from `pending`.

7. Responses and user-visible outcomes

Successful booking

After both records are committed, redirect to `/bookings` and show this success message:

```
Booking created. Payment is pending; no payment has been processed.
```

Invalid form values

If the pet ID or service ID is not a positive integer, or the date is missing, malformed, not a real date, or earlier than today, do not create either record. Redirect to `/services` and show:

```
Choose a pet, service, and valid booking date.
```

Pet or service unavailable

If the selected pet does not belong to the customer, or the service does not exist or is inactive, do not create either record. Redirect to `/services` and show:

```
That pet or service is unavailable for booking.
```

Database failure

If booking creation fails, log the error, do not report success, and redirect to the service form with:

```
Unable to create the booking. Please try again.
```

If loading pets or services fails, return HTTP `500` with an appropriate unavailability message. If loading Booking History fails, return HTTP `500` with `Unable to load booking history.`

Booking History

Show only bookings whose pet is owned by the signed-in customer, ordered newest first. For each booking, display the booking ID, date, pet name, service name, booked price, booking status, and payment status. If no payment row exists, display `No payment record`.

8. Acceptance criteria

AC1: Create a booking for an owned pet and active service

- **Given** Clara is signed in, owns Mira, and Basic Bath is active,
- **when** Clara submits Mira's pet ID, Basic Bath's service ID, and a valid booking date,
- **then** the app creates a booking for Mira and Basic Bath with status `pending`,
- **and** saves Basic Bath's price as `booked_price`,
- **and** creates one related payment for that booked price with status `pending`,
- **and** redirects Clara to Booking History with the successful-booking message,
- **and** does not process a payment.

AC2: Accept today's date and future dates

- **Given** the submitted date is a real date equal to or later than today,
- **when** the customer submits a booking,
- **then** the date is accepted, provided the pet and service are valid.

For the example scenario, `2026-11-03` is valid when submitted on or before 3 November 2026. Once the current date is after that date, the example booking date must be rejected.

AC3: Reject invalid dates and missing or invalid IDs

- **Given** the date is missing, malformed, impossible (for example `2026-02-30`), or earlier than today, or either ID is missing or not a positive integer,
- **when** the customer submits the form,
- **then** the app redirects to `/services` with the validation error,
- **and** creates neither a booking nor a payment.

AC4: Reject another customer's pet

- **Given** a customer submits a pet ID that they do not own,
- **when** the app attempts to create the booking,
- **then** it rejects the request with the unavailable pet/service error,
- **and** creates neither a booking nor a payment.

AC5: Reject an inactive or unknown service

- **Given** the submitted service ID does not identify an active service,
- **when** the app attempts to create the booking,
- **then** it rejects the request with the unavailable pet/service error,
- **and** creates neither a booking nor a payment.

AC6: Keep booking and payment writes atomic

- **Given** either the booking insert, payment insert, or transaction commit fails,
- **when** the app handles the failure,
- **then** it rolls back the transaction,
- **and** does not show the success message or leave a partial booking/payment pair.

AC7: Show only the customer's booking history

- **Given** a signed-in customer has bookings,
- **when** they open `/bookings`,
- **then** the app displays only bookings for pets they own, newest first,
- **and** shows the booking and payment statuses for each row.

AC8: Require login

- **Given** a visitor is not signed in,
- **when** they open `/services`, submit to `/services/book`, or open `/bookings`,
- **then** the app redirects them to `/login`.

9. Open questions and deferred decisions

There are no unresolved decisions for the baseline acceptance checks above.

The following are discussion points for a later revision, not additional requirements for this exercise:

- Should malformed IDs receive a distinct error message from unavailable pets or services?
- Should booking dates be restricted by a maximum advance-booking window?
- Should the app prevent duplicate bookings for the same pet, service, and date?
- What booking and payment status transitions should follow a real payment provider response?
- Should booking history include cancelled bookings or payment references?

Do not silently add these behaviours. Agree on any change and update the specification first.

10. Verification checklist

Use the signed-in app in a browser. Inspect the database to confirm the booking and payment rows and their values. These are expected results, not evidence that checks have already been executed.

Check	Request or action	Expected result
AC1	Sign in as Clara; book Mira for Basic Bath on a valid date	Redirect to `/bookings`; success message; one pending booking and one pending payment with matching price

Check	Request or action	Expected result
AC2	Submit today's date and a future date	Both accepted when all other booking details are valid
AC3	Submit a past, malformed, impossible, or missing date	Validation error; no booking or payment created
AC3	Submit a missing, non-numeric, zero, or negative ID	Validation error; no booking or payment created
AC4	Submit a pet ID owned by another user	Unavailable pet/service error; no booking or payment created
AC5	Submit an inactive or unknown service ID	Unavailable pet/service error; no booking or payment created
AC6	Simulate a database failure during payment insertion	Transaction rolled back; no partial records or success message
AC7	View `/bookings` as two different customers	Each sees only bookings for their own pets
AC8	Open each booking route while signed out	Redirect to `/login`
Regression	Repeat an existing login and journals flow	Existing behaviour remains unchanged

Record your own actual results:

Check	Actual result and evidence	Pass / Fail
AC1	Not run	Not assessed
AC2	Not run	Not assessed
AC3	Not run	Not assessed
AC4	Not run	Not assessed
AC5	Not run	Not assessed
AC6	Not run	Not assessed
AC7	Not run	Not assessed
AC8	Not run	Not assessed
Regression	Not run	Not assessed